

Mikroelektronische Schaltungen und Systeme

Vorlesung 14 Timing sequentieller Logik

Prof. Jürgen Becker
Institut für Technik der Informationsverarbeitung (ITIV)



Mikroelektronische Schaltungen und Systeme

Lect.1

Intro/CMOS Basics

Lect.2

Single Stage
Amplifiers

Lect.3

Noise

Lect.4

Differential Amplifiers

Lect.5

Current Mirrors

Lect.6

Frequency
Response

Lect.7

Feedback

Lect.8

Operational Amplifiers

Lect.9

Stability and
Frequency
Compensation

Lect.10

Switched-Capacitor
Circuits

Lect.11

Introduction to Data
Converters

Lect.12

Combinational Logic
Circuits

Lect.13

Sequential Circuits

Lect.14

Timing of Sequential
Circuits

Lect.15

Digital Building
Blocks

Lect.16

Memory

Lect.17

Technology
Mapping

Lect.18

FPGA

Lect.19

Physical Design I

Lect.20

Physical Design II

Lect.21

Q&A

1. Dynamic Discipline
2. System Timing
3. Clock Skew
4. Metastabilität
5. Synchronisierer
6. Parallelität



Dynamic Discipline

1

Dynamic Discipline

Clock-to-Q Contamination und Propagation Delay

- Die **Definitionen** für delays bei **sequentieller Logik** entsprechen denen für kombinatorische Logik. Sie werden als **Clock-to-Q-Contamination** und **Propagation Delay** bezeichnet.

	Kombinatorische Logik	Sequentielle Logik
Referenzpunkt	Änderung eines beliebigen Inputsignals (gemessen an 50% Schwelle)	Ansteigende Flanke des Taktsignals (gemessen an 50%-Schwelle)
Endpunkt	Übergang des Outputs (gemessen an 50 % Schwelle)	Übergang des Outputs (gemessen an 50 % Schwelle)

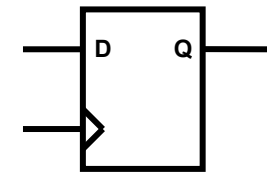
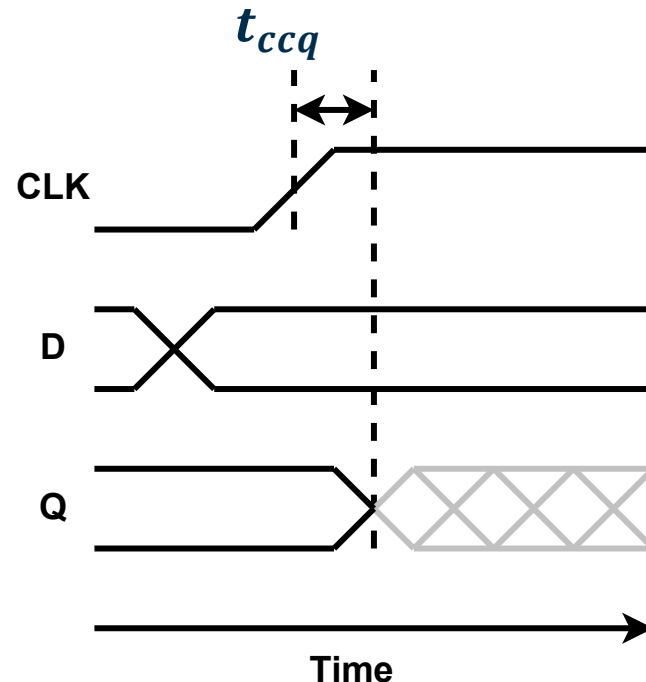
Anmerkung: Das **Q** bezieht sich auf die **Standardbezeichnung** für den **Output** eines Flipflops.

Siehe S. 142 [1]

Dynamic Discipline

Clock-to-Q Contamination Delay

- Analog zum contamination delay t_{cd} in der kombinatorischen Logik bezieht sich der **clock-to-Q contamination delay** t_{ccq} auf die **Mindestzeit** zwischen der steigenden Flanke der Clock und der **ersten** Outputänderung für sequentielle Logik.

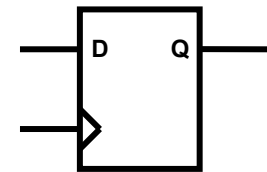
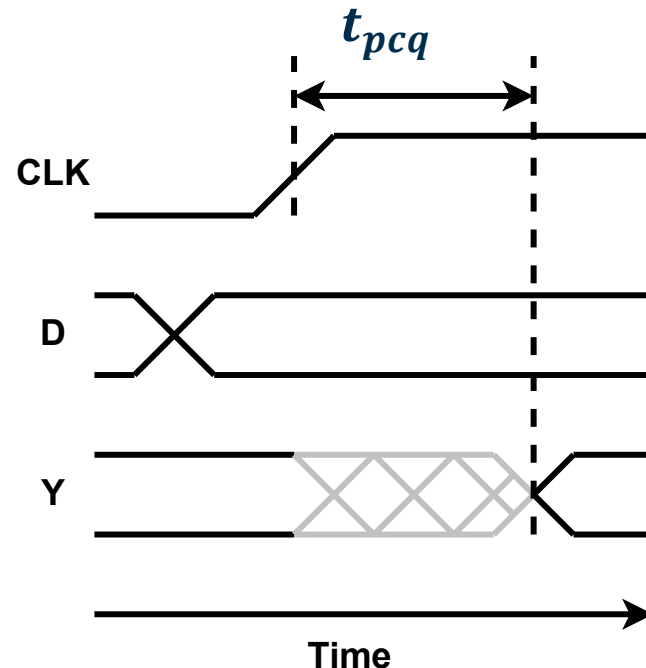


Referenz S. 142 [1]

Dynamic Discipline

Clock-to-Q Propagation Delay

- Analog zum propagation delay t_{pd} in der kombinatorischen Logik bezieht sich der clock-to-Q propagation delay t_{pcq} auf die maximale Zeit zwischen der steigenden Flanke des Takts und der endgültigen Outputänderung für die sequentielle Logik.

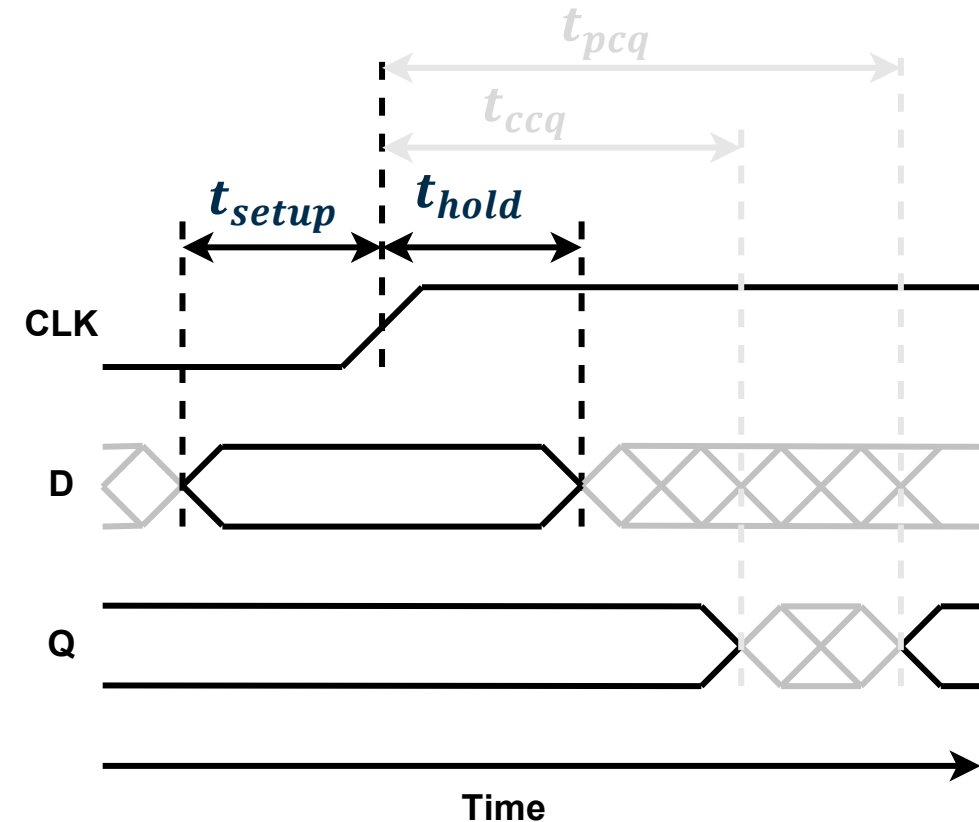


Referenz S. 142 [1]

Dynamic Discipline

Setup und Hold Time

- **Setup Time:** Alle Inputsignale müssen mindestens t_{setup} vor der steigenden Flanke des Clocksignals stabil sein.
- **Hold Time:** Alle Inputsignale müssen ihren Wert mindestens t_{hold} lang nach der steigenden Flanke des Taktsignals halten.



Siehe S. 142 [1]

Dynamic Discipline

Setup und Hold Time

➤ Dynamic Discipline:

Alle Inputs einer synchronen sequentiellen Schaltung müssen während der **Setup- und Hold-Zeiten** in Bezug auf die Flanken des Clocksignals stabil sein.

- Da das Timing der sequentiellen Logik durch die **Flanken (edges)** des Clocksignals bestimmt wird, muss jedes **Inputsignal** entsprechend agieren, damit die sequentielle Schaltung es korrekt **abtasten** kann.

Siehe S. 142 [1]



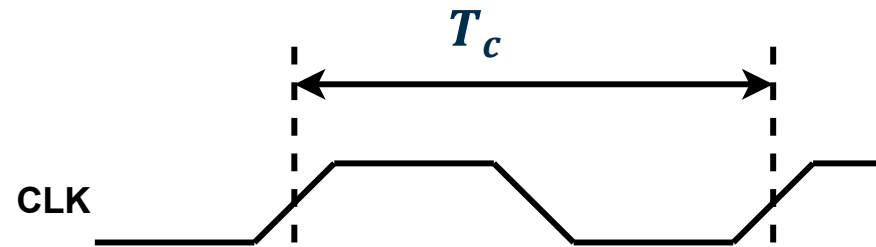
System-Timing

2

System-Timing

Clock Frequenz

- Die **Clock Period (Taktperiode)** T_c ist die Zeit zwischen den steigenden Flanken eines Clocksignals (Taktsignal).
- Die **Clock Frequency (Taktfrequenz)** f_c ist der Kehrwert der Periode. $f_c = 1/T_c$

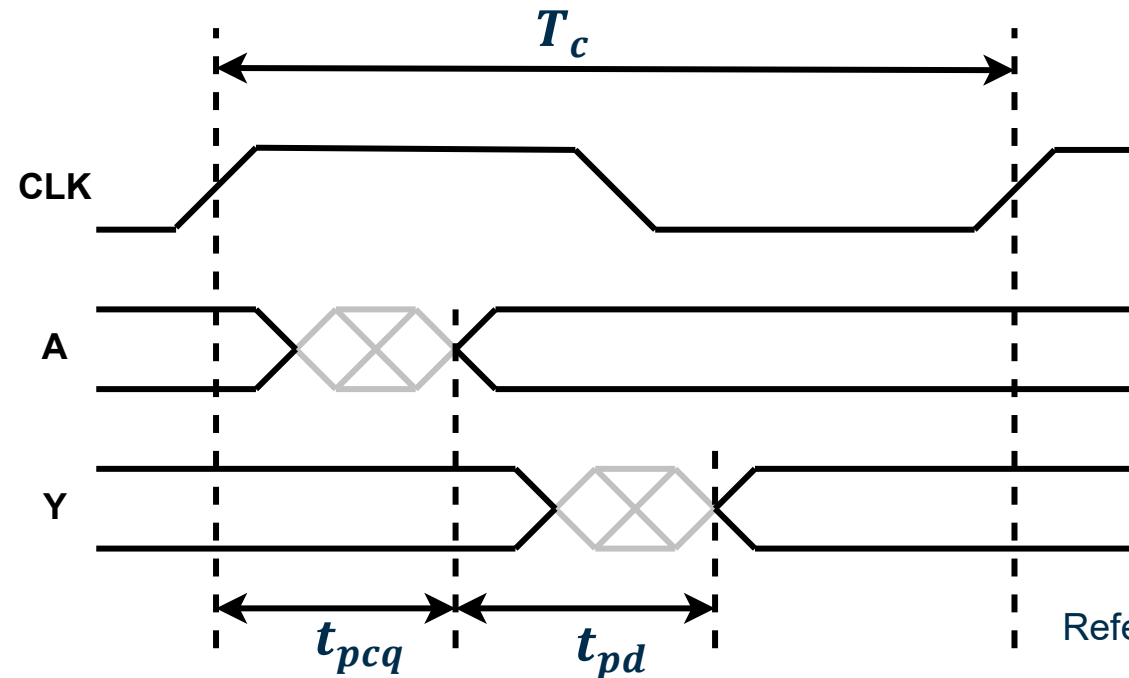
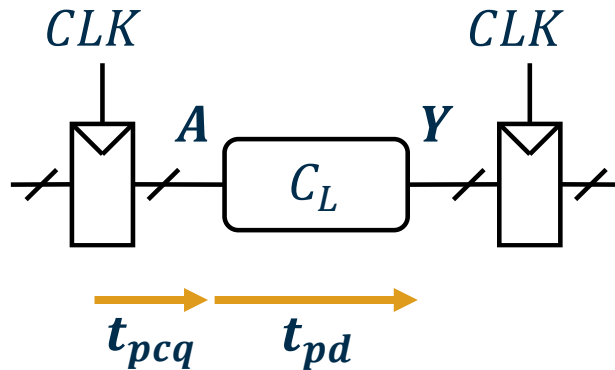


Referenz S. 142 [1]

System-Timing

Setup Time Constraint

- Innerhalb eines Clockzyklus muss jede Signalkette den folgenden Weg nehmen.
 - Ausbreitung durch das Register. (t_{pcq})
 - Ausbreitung durch die kombinatorische Logik. (t_{pd})



Referenz S. 143 [1]

System-Timing

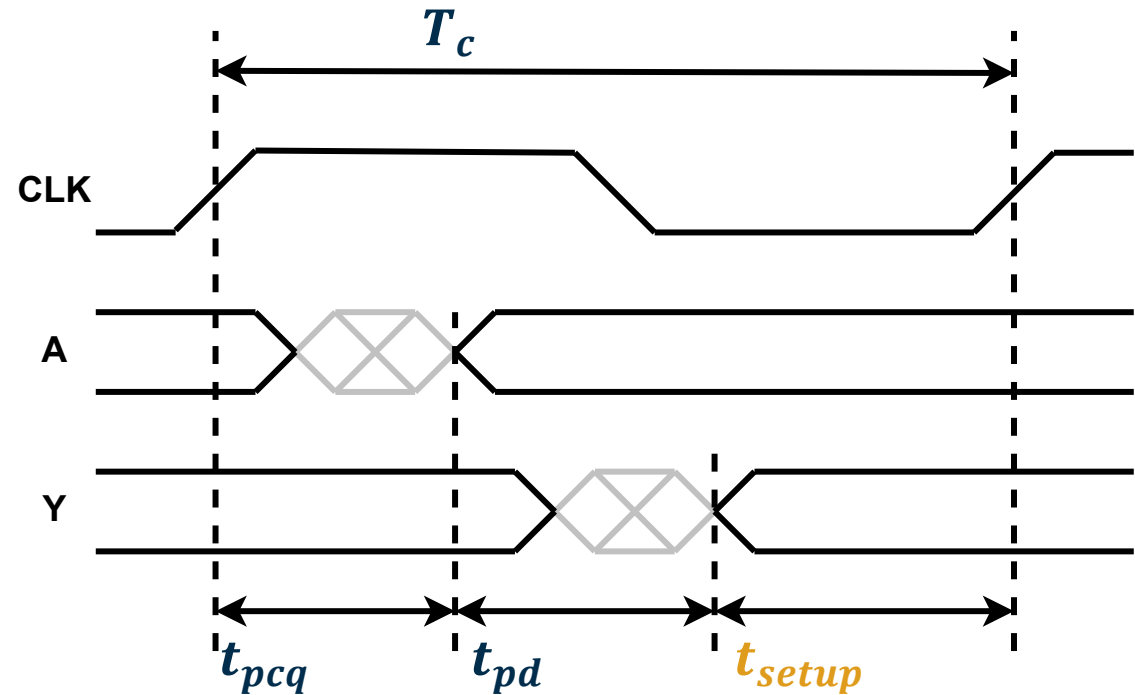
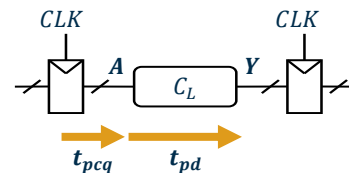
Setup Time Constraint

- Nach Durchlaufen der Signalkette muss sich der Output der kombinatorischen Logik vor der **setup time** in Bezug auf die nächste Taktflanke stabilisiert haben.

- Dies führt zu folgender Gleichung:

$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$

Diese legt eine minimale Clock Period fest.



Referenz S. 143 [1]

System-Timing

Setup Time Constraint

- Der **clock-to-Q propagation delay** von Flipflops und die **setup time** sind technologieabhängig. Auch die **Taktperiode** ist eine Designentscheidung, die für das jeweilige Design vordefiniert ist.
- Die **propagation delay** der kombinatorischen Logik ist der einzige Parameter, der optimiert werden muss.
 - Die umgestellte Gleichung
$$t_{pd} \leq T_c - (t_{pcq} + t_{setup})$$
gibt die **Obergrenze** für den **propagation delay** für die kombinatorische Schaltung an.
- Der Term $t_{pcq} + t_{setup}$ wird als **sequencing overhead** bezeichnet.

Siehe S. 143 [1]

System-Timing

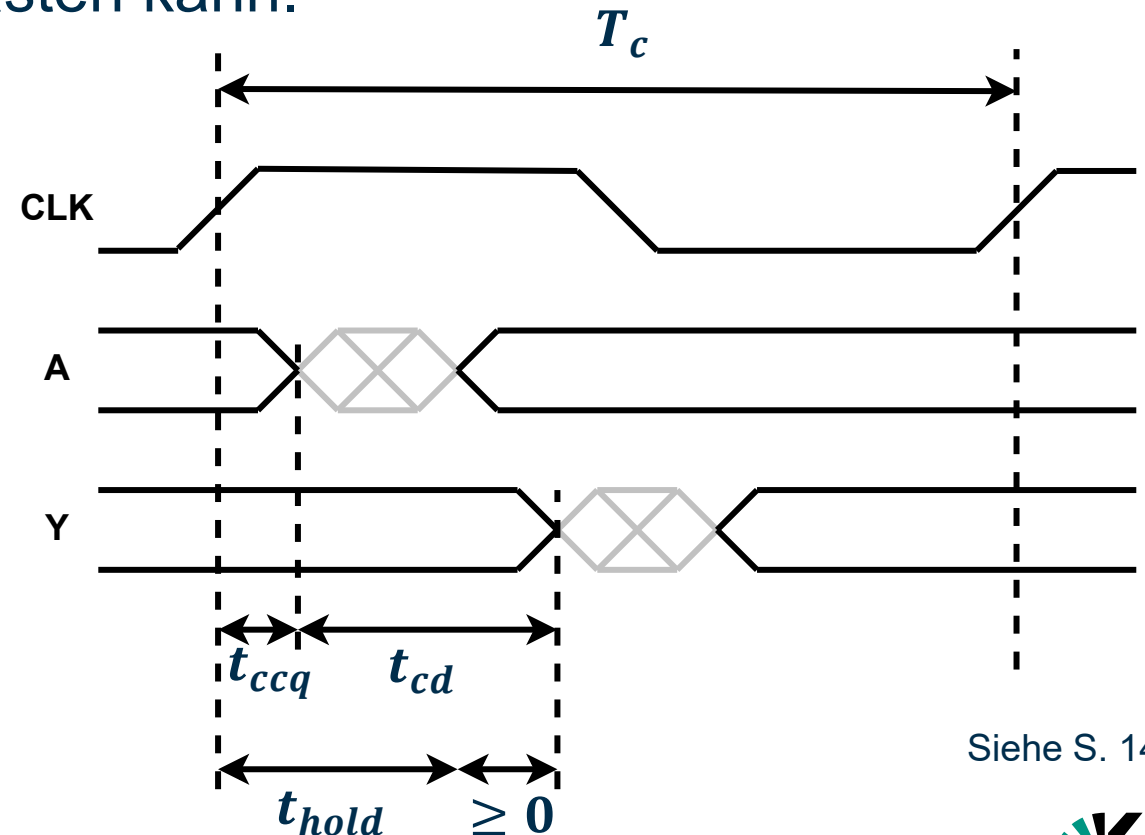
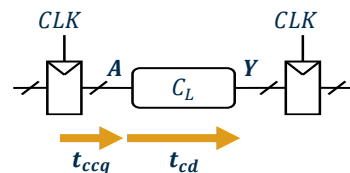
Hold Time Constraint

- Die erste mögliche Änderung des Outputsignals der kombinatorischen Logik muss nach Ablauf der **hold time** des nach folgenden Registers erfolgen, damit das Register den **vorherigen Outputwert** abtasten kann.

➤ Dies führt zu folgender Gleichung:

$$t_{ccq} + t_{cd} \geq t_{hold}$$

Diese legt eine Mindestzeit fest, nach der sich der erste Wert am Output der kombinatorischen Logik ändern darf.



Siehe S. 144 [1]

System-Timing

Hold Time Constraint

- Der **clock-to-Q contamination** delay von **Flipflops** und die **Hold Time** sind **technologieabhängig**
- Der **contamination delay** der **kombinatorischen Logik** ist der **Parameter**, der berücksichtigt werden muss.

- Die umgestaltete Gleichung:

$$t_{cd} \geq t_{hold} - t_{ccq}$$

gibt die **Untergrenze** für den **zulässigen contamination delay** für die **kombinatorische Schaltung** an.

Siehe S. 144 [1]

System-Timing

Setup vs. Hold Time Constraint

Anmerkung:

- Verstöße gegen die **Setup Time constraints** können durch eine **Verlängerung** der **Taktperiode** behoben werden.
- Verstöße gegen die **Hold Time constraints** sind **weitaus bedenklicher**, da sie einen **höheren contamination delay t_{cd}** der **kombinatorischen Logik** erfordern, was nur durch ein **Redesign** der **Schaltung** möglich ist.

Referenz S. 145 [1]

System-Timing

Hold Time Constraint

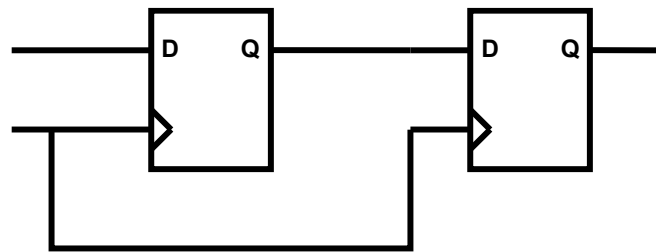
Anmerkung

Für **kaskadierte Register** vereinfacht sich die Gleichung zu

$$t_{ccq} \geq t_{hold}$$

Die Aussage $t_{cd} = 0$ gilt , da keine kombinatorische Logik dazwischen liegt.

- Ein Flip-Flop-Design muss einen größeren clock-to-Q contamination delay als seine hold time aufweisen.



Siehe S. 145 [1]



Clock Skew

3

Clock Skew

Einführung

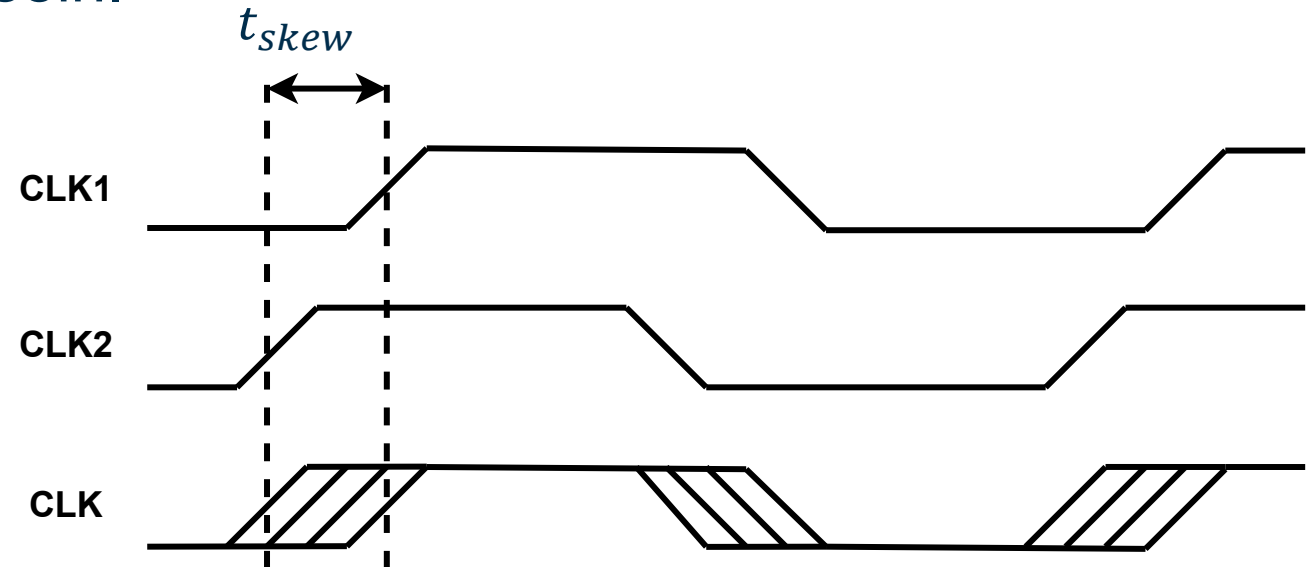
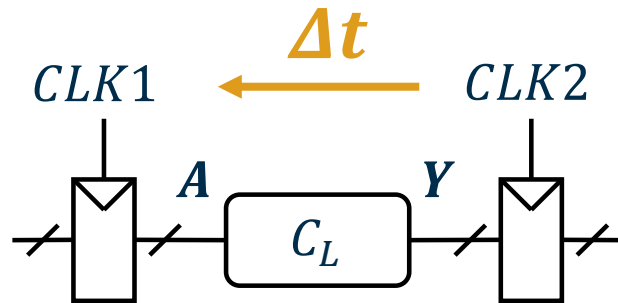
- Die Annahme, dass alle Komponenten in einem **Schaltkreis** die **Taktflanken gleichzeitig** empfangen, ist nur **ideal**.
 - In der **Realität** gibt es **Abweichungen** bei den **Clockflanken** aufgrund von:
 - **Verzögerungen** durch unterschiedlich lange **Leitungen zwischen** der **Clockquelle** und den **Registern**.
 - **Rauschen** auf den Clockleitungen.
 - **Clock Gating** (Das Leiten des Clocksignals durch komb. Logik, beispielsweise zur Implementierung eines Freigabemechanismus, führt zu Verzögerungen)
- Die **Abweichung** der Clockflanken wird als **Clock Skew (Taktversatz)** bezeichnet.

Referenz S. 148 [1]

Clock Skew

Worst Case

- Für die Timing-Analyse muss der **worst case** berücksichtigt werden.
- Je nach **Szenario** kann der ungünstigste Fall die **späteste** oder die **früheste** mögliche Taktflanke sein.



Referenz S. 148 [1]

Clock Skew

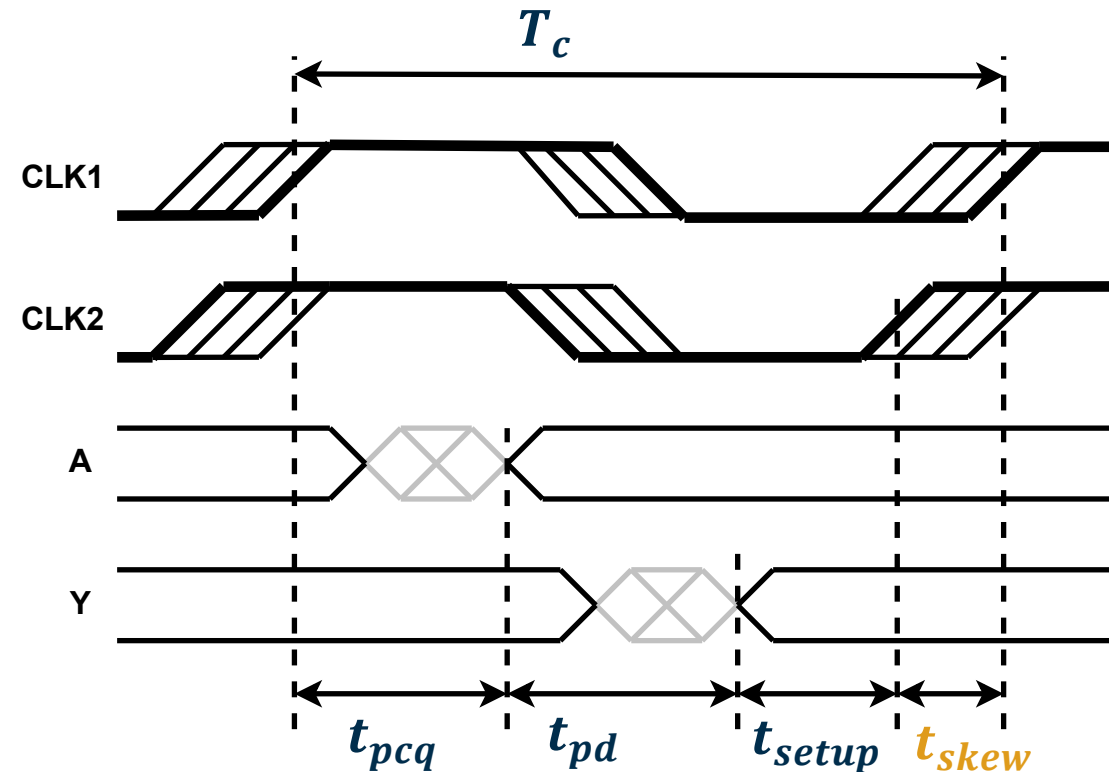
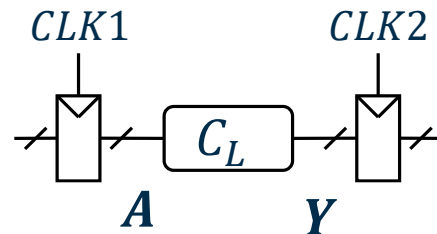
Setup Time Constraint (Clock Skew berücksichtigt)

- Beim **setup time constraint** ist der **ungünstigste Fall** die **späteste** versetzte **Taktflanke** für **Register 1** und die **früheste** versetzte **Taktflanke** für **Register 2**.

➤ Führt zu **folgenden Gleichungen**:

$$T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$$

$$t_{pd} \leq T_c - (t_{pcq} + t_{setup} + t_{skew})$$



Referenz S. 149 [1]

Clock Skew

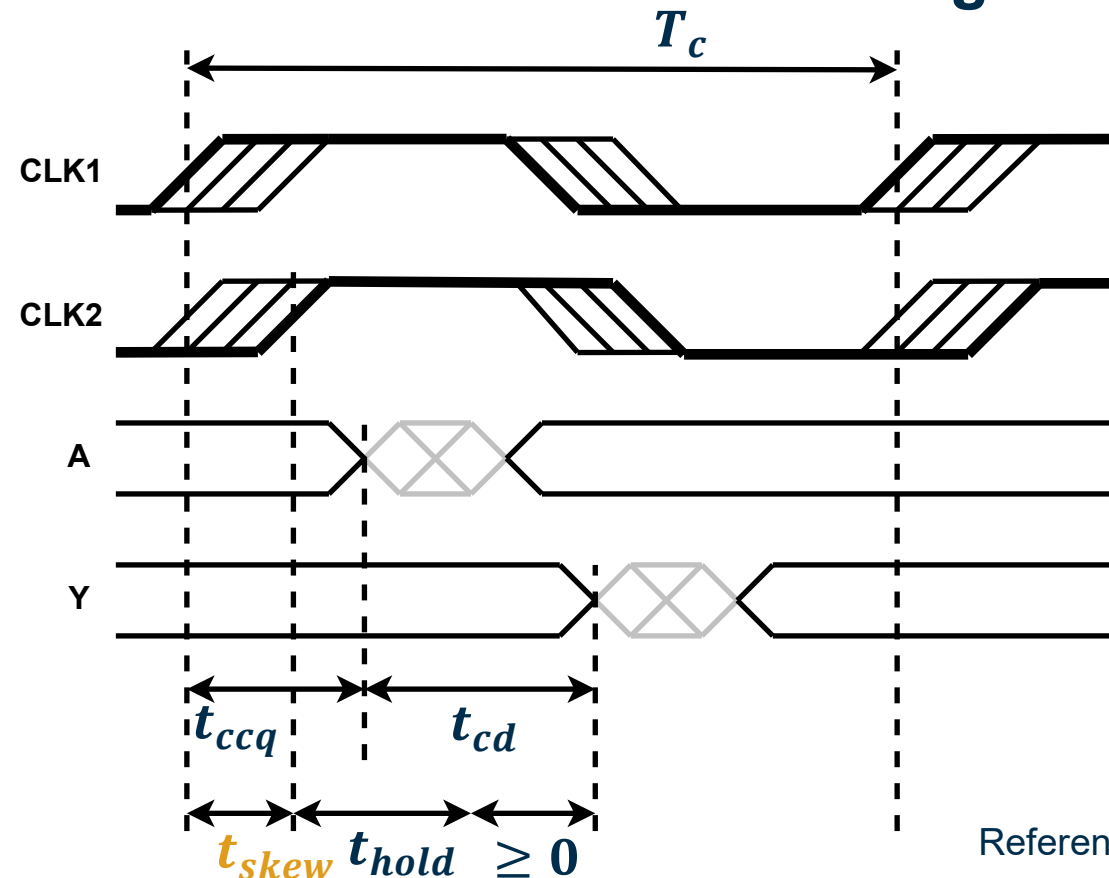
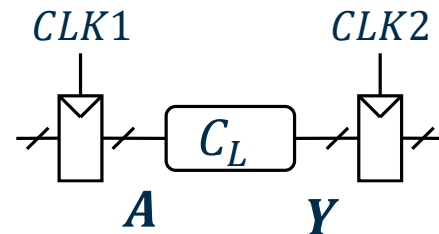
Hold Time Constraint (Clock Skew berücksichtigt)

- Beim **Hold Time constraint** ist der ungünstigste Fall die **früheste** versetzte **Taktflanke** für **Register 1** und die **späteste** versetzte **Taktflanke** für **Register 2**

➤ Führt zu folgenden Gleichungen:

$$t_{ccq} + t_{cd} \geq t_{hold} + t_{skew}$$

$$t_{cd} \geq t_{hold} - t_{ccq} + t_{skew}$$



Referenz S. 150 [1]



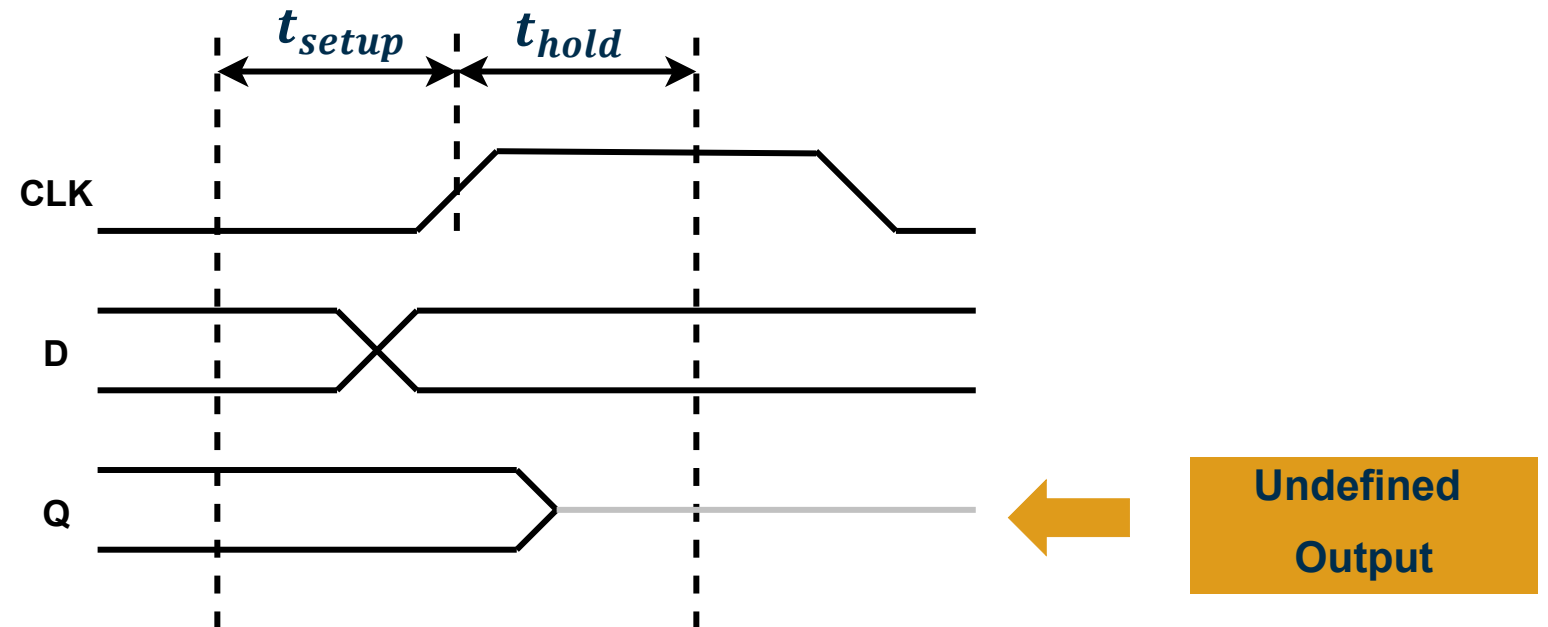
Metastabilität

4

Metastabilität

Einführung

- Verstöße gegen die *Dynamic Discipline* (Input-Änderungen während der *Setup*- und *Hold*-Zeiten) für einen Flipflop führen zu einem **undefinierten Output**.

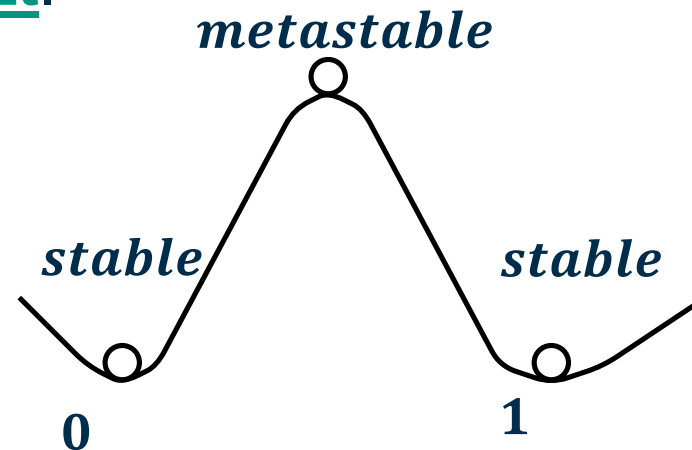


Referenz S. 151 [1]

Metastabilität

Metastabiler Zustand

- Ein **bistabiles** Gerät hat einen **metastabilen** Zustand zwischen zwei stabilen Zuständen.
- Bei **Verletzung** der **Dynamic Discipline** für einen Flipflop ist der Output nicht **unbekannt** (0 oder 1), sondern **undefiniert** und kann daher eine **Spannung zwischen 0 und V_{DD}** annehmen, ein **metastabiler** Zustand.
- Das **Flipflop** führt den **Output selbstständig** in einen stabilen Zustand, aber die **Zeit** (*resolution time*) ist **nicht begrenzt**.



Referenz S. 151 [1]

Metastabilität

Resolution Time

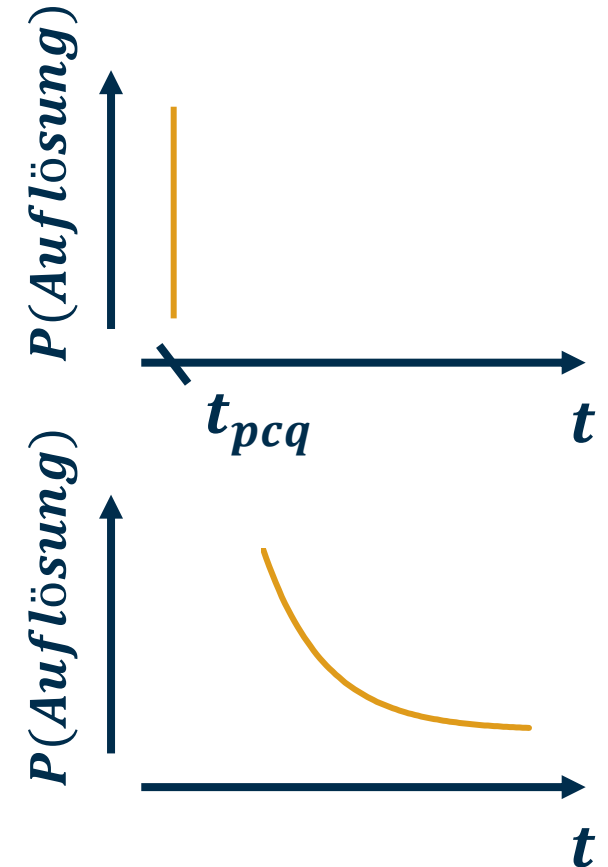
- Bei einer **rechtmäßigen Änderung** des **Inputsignals** entspricht die **resolution time** t_{res} der Übertragungsverzögerung zwischen Clock und Q:

$$t_{res} = t_{pcq}$$

- Bei einer **unzulässigen Verletzung** der Dynamic Discipline ist die **Auflösungszeit** t_{res} eine Zufallsgröße. Die **Wahrscheinlichkeit**, dass t_{res} eine **Zeit** t **überschreitet**, nimmt **exponentiell** mit t ab:

$$P(t_{res} > t) = \frac{T_0}{T_c} e^{-\frac{t}{\tau}} \quad , \quad t \gg t_{pcq}$$

- T_0 und τ sind **Eigenschaften eines Flipflops**.



Siehe S. 151+152 [1]



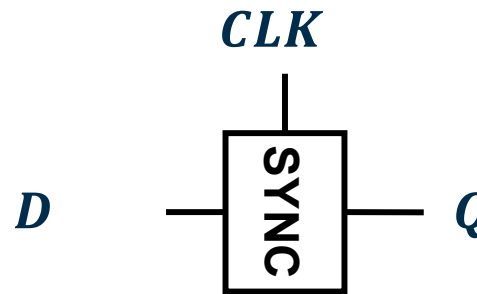
Synchronizer

5

Synchronizer

Einführung

- Um **metastabile Zustände** zu vermeiden, sollten alle asynchronen Inputs einen **Synchronizer** durchlaufen.



Referenz S. 152 [1]

Synchronizer

Einführung

- Eine mögliche Implementierung besteht aus **zwei in Reihe geschalteten Flipflops**:

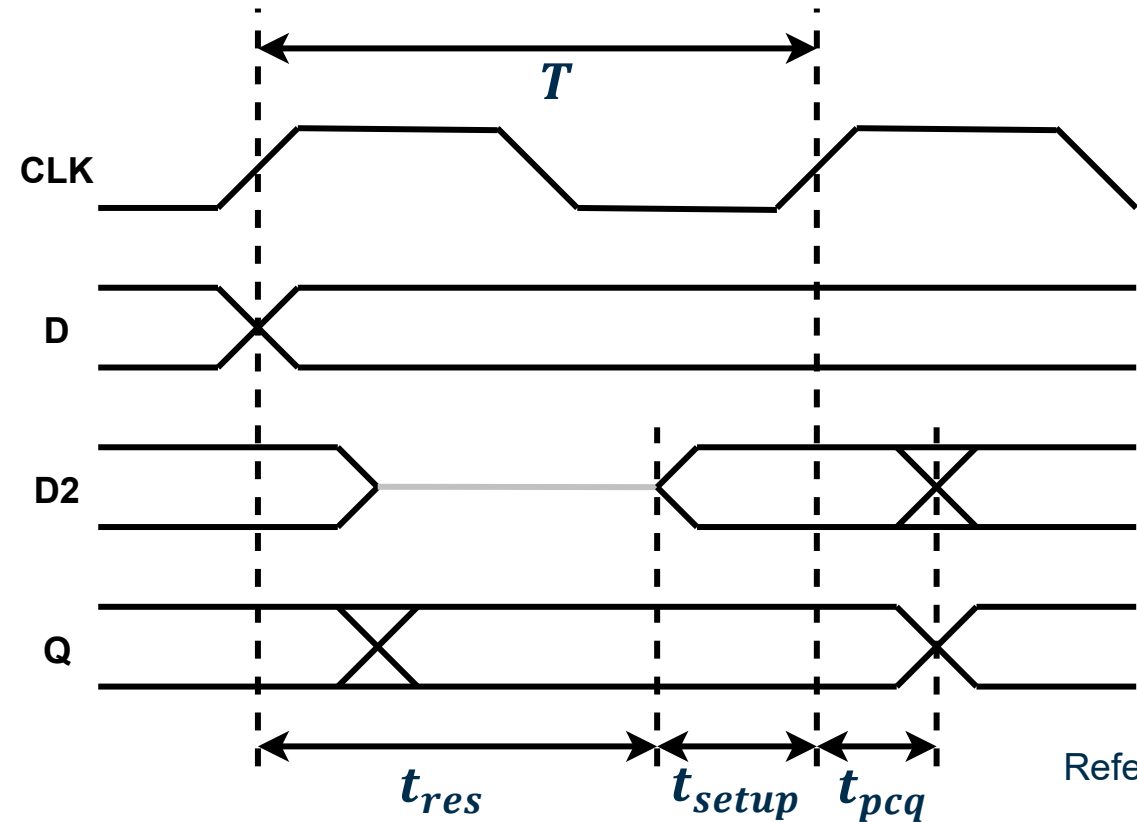
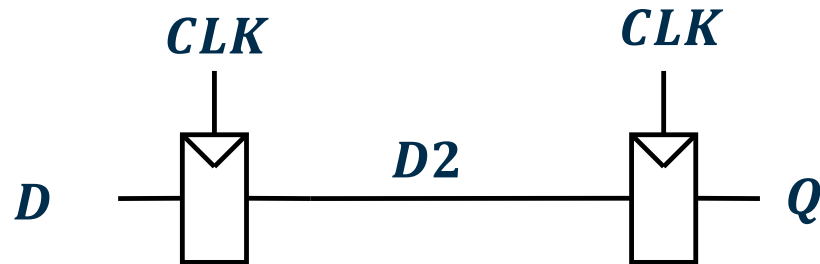


Referenz S. 153 [1]

Synchronizer

Implementierung

- Wenn das **erste Flipflop** für eine gewisse Zeit in einem **metastabilen Zustand** verbleibt, hat der **Output** des **zweiten Flipflops** immer noch einen **stabilen Zustand**



Referenz S. 153 [1]

Synchronizer

Mittlere Zeit zwischen Ausfällen: *Mean Time between Failure – MTBF*

- Dass der **zweite Flipflop** in einem **stabilen Zustand** bleibt, trifft zu, solange der **erste Flipflop** seinen Zustand bis zum **Beginn der Setup Time** für die nächste Taktflanke **auföst**. Die **Ausfallwahrscheinlichkeit** beträgt dann:

$$P(failure) = P(t_{res} > T_c - t_{setup}) = \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$$

- Wenn sich der **Input D** des **Synchronisierers N Mal pro Sekunde** ändert, beträgt die **Ausfallwahrscheinlichkeit pro Sekunde**:

$$P(failure)/sec = N \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$$

Referenz S. 153 [1]

Synchronizer

Mittlere Zeit zwischen Ausfällen: *Mean Time between Failure – MTBF*

- Die allgemeine Zuverlässigkeit eines Systems lässt sich mit der *Mean Time between Failure (MTBF)* angeben.
- Dies ist der **Kehrwert** der **Ausfallwahrscheinlichkeit pro Sekunde**:

$$MTBF = \frac{1}{P(failure)/sec} = \frac{T_c e^{\frac{T_c - t_{setup}}{\tau}}}{NT_0}$$

- Die *MTBF* verbessert sich **exponentiell** mit einer **längeren Taktperiode T_c** .
- Für die meisten Systeme reicht ein **Synchronizer** der **1 clk cycle wartet** für einen sicheren *MTBF*
- Bei **Hochgeschwindigkeitssystemen** kann es erforderlich sein, die Wartezeit auf ein **Vielfaches von T_c** zu erhöhen.

Referenz S. 153 [1]



Parallelität

6

Parallelität

Einführung

Definition:

Ein **Token** ist ein **Set** von **Inputs**, die zu einem **Set** von **Outputs** verarbeitet werden.

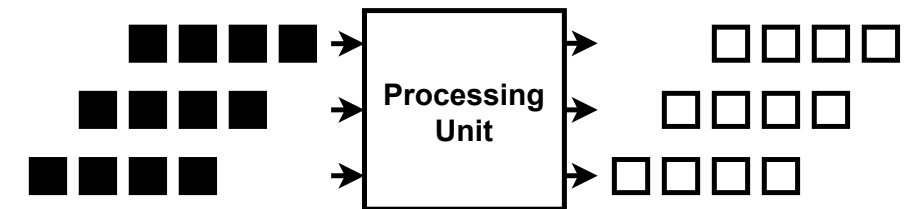
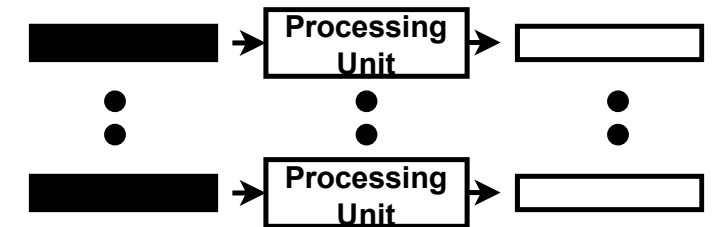
- Ein **System** kann **charakterisiert** werden **durch**:
 - **Latency (Latenz): Zeit**, die benötigt wird, um ein **Token** im **System** zu **verarbeiten**.
 - **Troughput (Durchsatz): Anzahl** der **Token**, die **pro Zeiteinheit** vom **System verarbeitet** werden.

Referenz S. 157–158 [1]

Parallelität

Räumliche vs. Zeitliche Parallelität

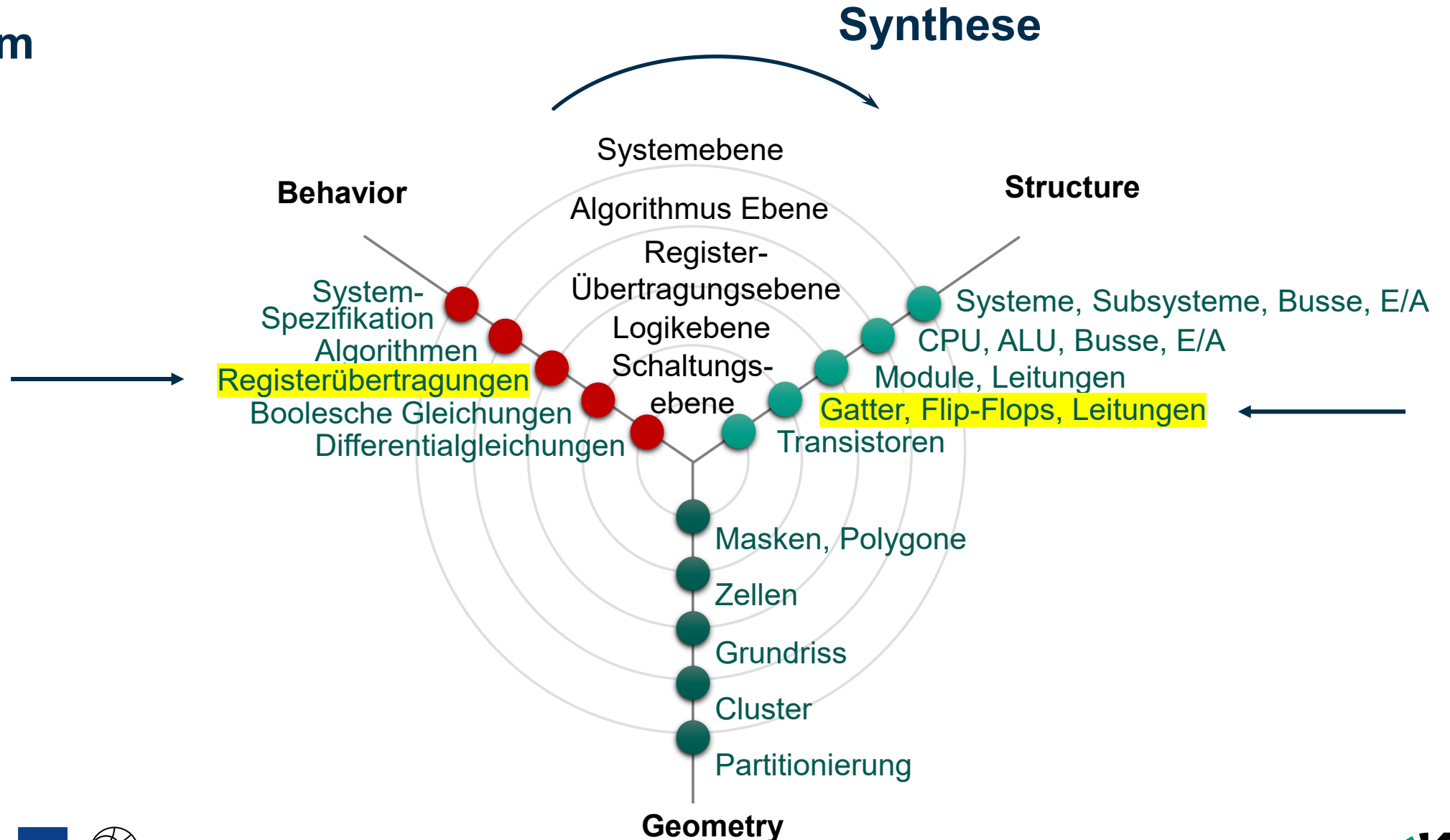
- **Parallelität** ist ein **Konzept** zur Verbesserung des **Durchsatzes** des **Systems** durch **Erhöhung** der Anzahl der **gleichzeitig** zu verarbeitenden **Token**.
- Es werden zwei **Konzepte** der **Parallelität** vorgestellt:
- ***Spatial* Parallelism**: Es gibt **mehrere Verarbeitungseinheiten**, die **mehrere Aufgaben gleichzeitig** verarbeiten können.
- ***Temporal* Parallelism**: Eine Aufgabe wird in **mehrere Phasen** unterteilt. Jede Aufgabe muss alle Phasen durchlaufen, aber **pro Phase** können **mehrere Aufgaben** verarbeitet werden, sodass sich die **Aufgaben überlappen**. Dieses **Konzept** wird gemeinhin als **Pipelining** bezeichnet.



Referenz S. 157–158 [1]

Advance Organizer

Y-Diagramm



Referenz

- [1] S. L. Harris und D. M. Harris, *Digitales Design und Computerarchitektur*, ARM®-Ausgabe. Waltham, MA: Morgan Kaufmann, 2016.

